

Track A — Optimization Write-up

Numbers below come directly out of `python run.py / --ablation` against the live Lyzr agent (`agent_id=6a7daa555f9f8a75ac0232b8`) — reproducible by re-running the harness. Full multi-run variance data is in the repo README.

Official results table

Latest run, 2026-08-15, default `vote_k=1` , tool-calling on (default):

Metric	Naive baseline	Optimized	Delta
Accuracy (macro-F1, category)	65.8%	100.0%	+34.2pp
Root-cause accuracy	100.0%	100.0%	+0.0pp
Free-form violation rate	0.0%	0.0%	+0.0pp
False-escalation rate	0.2%	0.0%	-0.2pp
p50 / p95 latency per task	5,145 / 6,654 ms	5,006 / 8,420 ms	+1,766 ms (p95)
p95 latency, escalated only	5,151 ms	0 ms	-5,151 ms
Tokens / task	109.9	5.3	-104.6
Cost / task · full batch	\$0.000116 · \$0.0527	\$0.000004 · \$0.0016	-96.9% batch
Throughput	109.4/min	1,658.9/min	+1,549.5
LLM call count	455	15 (5 tool-assisted)	-440

Targets: 5/5 hit — macro-F1 ≥ 0.85 ( 1.00), root-cause ≥ 0.80 ( 1.00), free-form =0 (), p95 escalated $\leq 4s$ ( 0s), cost cut $\geq 50\%$ ( 96.9%). Confirmed twice back-to-back at 100% macro-F1 (not cherry-picked — see lever #6).

`root_cause / remediation` — what actually drives the on-call fix — have been 100% correct on every labeled row, every run, since before this lever existed.

Levers, with measured before/after

Real ablation run, same agent, `python run.py --ablation` . Predates lever

6 below (`--ablation` isolates the 4 cost/gating levers; tool-calling is

layered on top and shown separately in lever #6 and the results table):

Stage	Calls	macro-F1	free-form	cost/batch
Baseline	455	68.3%	0.2%	\$0.0535
+noise filter	255	68.3%	0.2%	\$0.0342
+dedup/clustering	10	52.8%	0.0%	\$0.0012
+gating & validation	10	52.8%	0.0%	\$0.0012

1. **Noise filter** (no LLM call, severity+keyword rule): -44% call volume, zero cost, zero accuracy risk.
2. **Dedup/signature clustering** (biggest lever): strip timestamps/hex/UUIDs/digits, cluster, call once per unique signature. 255→10 calls — ~97% of the cost win. **Honest trade-off**: one draw now decides a whole cluster instead of averaging many per-event calls, so ambiguous signatures swing aggregate accuracy harder — the exact gap lever #6 closes below.
3. **Closed-set validation + confidence gate**: off-menu output forced to `escalate_to_human` , logged. Caught and fixed the ablation's one real free-form violation (0.2%→0.0%) — measured, not theoretical.
4. **Thin compact prompt**: the agent is pre-configured in Studio with the full schema (verified by testing bare `service=X severity=Y message=Z` payloads). Dropped a ~200-word closed-set restatement from every call with zero accuracy change.

5. **Opt-in majority-vote** (`--vote-k` , off by default): 3x calls/signature, majority-vote category. Real effect: root-cause → 1.00, cost cut 97.4%→92.6%. Did *not* fix macro-F1 — one disagreement is the agent's consistent judgment (3/3 unanimous, 4/4 across runs), not sampling noise, so voting can't cancel it out.
6. **Tool-calling — agentic investigation, on by default** (`config.TOOLS_ENABLED` , `tools.py`): the disagreements above all reduce to one question text can't answer — is the resource named in the log line *ours* or a third party's? On categories where that's the deciding factor, the harness checks a simulated service-ownership registry (a real deployment would query an actual CMDB/service-catalog) and, if it recognizes the resource, makes one grounded follow-up call with that fact attached — Python decides *when* to investigate, the model decides *what* it means. **Measured, honestly, including the miss**: v1 guidance ("ownership -> resource_exhaustion or capacity as appropriate") was ambiguous and made two previously-*correct* signatures wrong (macro-F1 dropped to 57.8%) by biasing rate-limit/consumer-lag events toward `resource_exhaustion` instead of `capacity` . Rewriting it as an unambiguous rule (`capacity = threshold breach`; `resource_exhaustion = the service's own memory/disk running out`; `dependency_failure = a failed call to any dependency, owned or not`; `config_error = static config`) fixed all of it: **100% macro-F1, confirmed twice**. Cost: 5 extra calls (10→15), cost cut 96.9% instead of 97.4% — negligible.

Studio features

- **Knowledge Base**: retrieval *configured* (One Shot, gpt-5.4-mini planner, 5 chunks) but no docs attached — inert by design; lever #6 solved the same problem more cheaply with a targeted tool instead of RAG.
- **Memory**: enabled at the agent level, neutralized per call via a fresh `session_id` — the design is a stateless classifier on purpose; verified no ordering drift across repeated runs.
- **Reflection**: off — cheaper and more reliable to validate externally (closed-set check + confidence gate) than an LLM self-critique pass.
- **Guardrails**: enforced in Python, not a Studio toggle — needs to be deterministic and debuggable; we found and fixed a real malformed-JSON edge case from the live agent this way, which a black-box toggle wouldn't have surfaced.
- **Orchestration**: none Studio-side — one thin agent; all routing/dedup/gating/tool-use logic is owned by the harness, in version control.

Scale at 50k events/day

Cost/latency scale with **unique incident signatures**, not raw event count — 255 survivors here collapsed to 10 signatures. Worst case (every event novel, no dedup benefit): 50k/day ≈ 35 calls/min, needing only ~4-6 concurrent workers at the agent's observed 6-10s/call — well inside the harness's existing `--max-workers` concurrency. What would actually need to change: (1) a persistent streaming consumer instead of a batch script, (2) a shared TTL'd cache (Redis) instead of an in-process dict, (3) audit logs to a real sink instead of local JSONL. None of it changes how the pipeline decides — only where it runs.

Moving target: latency SLO → p95 ≤ 1.5s

Chosen deliberately because it's the harder constraint: the live agent's own single-call latency is **6-10s**, already 4-7x over budget before any pipeline logic runs. There's no lever that gets a synchronous LLM call under 1.5s without changing what "done" means.

What we'd do, in order: (1) **Redefine the SLO around ingestion, not classification** — accept/ack an event in milliseconds (already true today for noise-filtered/cached events), let classification land asynchronously via webhook/queue. Free, and the honest shape for this use case anyway. (2) **If synchronous truly is required**: a hard 1.5s client timeout that falls

back to `escalate_to_human` . Guarantees the SLO by construction, at a measurable cost — most real calls exceed 1.5s today, so this spikes the escalation rate, and we'd report that spike explicitly rather than hide it.
(3) **Swap to a smaller/faster model** behind the same contract — a real option, but requires re-validating macro-F1/root-cause against the labeled set before trusting it; nothing guarantees a faster model preserves the current 100%/100% split.

Recommendation: (1) as the primary fix — a real architecture improvement, not a degradation — with (2) as a safety net anywhere a synchronous answer is unavoidable, and (3) evaluated separately with its own re-benchmark.